

PHASE 2 — CRM Opportunity Commands & API

Status: BINDING IMPLEMENTATION SPECIFICATION

Version: 2.0 - Consolidated Phase 2 Planning Contract

Scope: Enterprise Business Platform / CRM Opportunity Application Commands, Minimum Queries and HTTP API

Operating principle: FOUNDATION-FIRST, NOT MVP-FIRST

This document consolidates the original Phase 2 specification with the approved authorization, field-security, policy/approval, pipeline-entry, update, idempotency/concurrency, and frontend-capability clarifications. Where this document repeats a previously binding rule, the stricter interpretation applies.

Normative language

- **MUST / MUST NOT:** binding requirement.
- **SHOULD / SHOULD NOT:** expected default; deviation requires an explicit repository-backed reason.
- **MAY:** optional and context-dependent.
- **OPEN DECISION:** must be surfaced for approval before implementation.

We are continuing the Enterprise Business Platform implementation. This specification is dedicated ONLY to **PHASE 2 - CRM Opportunity Commands & API**.

Previous phases are completed:

- Phase 0.5 — MasterData / Party Foundation 
- Phase 1 — CRM Lifecycle / Pipeline Foundation 
- Phase 1.5 — Enterprise Access / Authorization Foundation 

Do NOT redesign or reopen those phases unless you find a concrete contradiction that blocks Phase 2.

Our architectural principle remains:

FOUNDATION-FIRST, NOT MVP-FIRST.

We do not need every future CRM capability today, but we must not introduce temporary shortcuts that force us to break the architecture later.

0. FIRST RULE — INSPECT BEFORE CHANGING

Do NOT start implementing immediately.

First inspect the actual repository state.

Read:

1. Current CRM implementation
2. Current CRM tests
3. Current migrations
4. MasterData / Party implementation
5. Pipeline / lifecycle implementation
6. Access / Authorization implementation completed in Phase 1.5
7. Relevant architecture docs
8. Current-state CRM analysis, especially if present:
docs/architecture-analysis/CRM_CURRENT_STATE_ANALYSIS.md
9. The binding CRM target model:
Enterprise_CRM_Target_Model_Binding_Implementation_Specification.pdf
10. Any Phase 1 / Phase 1.5 implementation reports or decision documents.

Do not trust old plans over actual code.

The repository is the source of truth for what is implemented.

The binding architecture documents are the source of truth for intended behavior.

If code and binding decisions conflict, REPORT the conflict before changing anything.

0A. REFERENCE PRECEDENCE + CONFLICT HANDLING — BINDING

Use the following interpretation rules during Phase 2 planning:

1. The actual repository is the source of truth for what is currently implemented.
2. Approved binding architecture/decision documents are the source of truth for intended behavior.
3. This Phase 2 specification is binding for Phase 2 scope, planning discipline, safety constraints and stop conditions.

4. Later explicitly approved decision records override earlier planning assumptions only for the exact decision they settle.

5. Old implementation plans, stale prompts, comments or inferred intent MUST NOT override actual code or binding decisions.

For every material conclusion in the Phase 2 plan, cite the concrete repository evidence:

- file path
- relevant type/class/module/function where applicable
- migration/table/policy/test where applicable
- binding decision document section where applicable

If a mandatory reference is missing:

- do not invent its content,
- mark the missing reference as a BLOCKER or OPEN DECISION depending on impact,
- continue inspecting what can be verified,
- final readiness MUST be NO if the missing reference prevents safe implementation.

Do not use external web research to replace missing repository or binding architecture evidence.

1. PHASE 2 GOAL

Phase 2 turns the existing CRM domain foundation into an actual usable application/API flow.

The target Opportunity lifecycle is conceptually:

Draft

v

Open

v

Won / Lost

IMPORTANT:

Lifecycle Status and Pipeline Stage are NOT the same concept.

Lifecycle is platform-level canonical state.

Pipeline Stage is configurable according to:

Sector Template

v

Tenant Configuration

v

Opportunity Pipeline

A tenant may have:

- 1 stage,
- 2 stages,
- 3 stages,
- or many stages.

Examples:

Furniture retail:

Open

- Bekliyor
- Teklif Verildi
- Görüşülüyor

Technical services:

Open

- Talep Alındı
- Keşif
- Teklif
- Onay Bekleniyor

Simple retail:

Open

- Bekliyor

All of these still map to the same canonical lifecycle:

Draft -> Open -> Won / Lost

2. DO NOT CONFUSE PIPELINE WITH QUOTE

A pipeline stage called:

"Teklif Verildi"

does NOT mean Quote is owned by CRM.

Pipeline stage is only the tenant's process representation.

Future Sales domain will own:

- Quote
- Quote Version
- Quote Lines
- Pricing
- Discount
- Quote public link
- Quote viewed/downloaded/commented
- Quote accepted/rejected
- Order

Therefore:

CRM Opportunity

!=

Quote

and:

"Quote Sent"

!=

Opportunity Won

A tenant may send a Quote during ANY Open pipeline stage if policy allows it.

Do NOT hard-code Quote behavior into a specific CRM pipeline stage.

Sales / Quote implementation is NOT part of this phase unless existing binding documents explicitly say otherwise.

3. PHASE 2 BUSINESS CAPABILITIES

Inspect the existing implementation and determine the correct application commands required to expose the lifecycle safely.

Expected conceptual capabilities include:

- Create Opportunity
- Open Opportunity
- Change Pipeline Stage
- Win Opportunity
- Lose Opportunity
- Reassign Opportunity, if already required by the target model

Do NOT blindly create these exact class names.

First reconcile them with the existing domain model and current code.

For example, if Create currently produces an Open opportunity rather than Draft, identify that conflict first.

For every command determine:

- Preconditions
- Authorization requirement
- Allowed lifecycle state
- Pipeline requirements
- Required fields
- Business invariants
- Idempotency requirement
- Evidence requirement
- Outbox event requirement
- Optimistic concurrency behavior
- Error model

4. AUTHORIZATION — PHASE 1.5 MUST BE USED

Phase 1.5 is finished.

Phase 2 MUST consume the existing authorization architecture.

Do NOT create a second local authorization mechanism inside CRM.

Every mutation must go through the established authorization/policy infrastructure.

Determine the appropriate capabilities/actions, such as conceptually:

crm.opportunity.create
 crm.opportunity.open
 crm.opportunity.change_stage
 crm.opportunity.win
 crm.opportunity.lose
 crm.opportunity.reassign

These are examples only.

Use the conventions already established by Phase 1.5.

Check:

- Principal
- Tenant
- ownership
- team/org scope
- territory scope if applicable
- action authorization
- record access

Do not invent a parallel permission model.

4A. AUTHORIZATION EVALUATION ORDER — BINDING

Do NOT model authorization as a single:

Authorization

-> Load Aggregate

step.

Phase 1.5 may contain multiple authorization layers.

For mutations, inspect and preserve the established architecture using the following conceptual order:

Authenticated Principal

v

Resolve Tenant / Organization Context

v

Coarse Capability / Action Authorization

v

Tenant-safe Aggregate Load

v

Record-Level Policy Evaluation

v

Field-Level WRITE Authorization

v

Policy Obligations / Approval Requirements

v

Domain Command

v

Transaction

The exact implementation MUST follow Phase 1.5 conventions.

Do not create a parallel CRM authorization system.

IMPORTANT:

Record-level authorization may require aggregate attributes such as:

- owner
- assigned principal
- team
- organization unit
- territory
- lifecycle state
- business amount
- other policy-relevant attributes

Therefore, where required:

coarse authorization

- > tenant-safe record load
- > record-level authorization

is expected.

The tenant-safe load MUST NOT expose cross-tenant existence through errors or timing-sensitive application behavior.

Policy evaluation may require BOTH persisted aggregate state and the proposed mutation/request values.

Therefore, where applicable, authorization/policy evaluation MUST be able to reason over:

Principal

- + Tenant / Organization Context
- + Current Aggregate
- + Proposed Change
- + other established policy context

Do not authorize only the current record state when the decision depends on the requested new values.

Examples:

- EstimatedAmount: 100,000 → 500,000 may cross an approval threshold.
- Reassignment to another team may change required scope or approval.
- A lifecycle outcome may be allowed for one amount/territory combination but approval-bound for another.

Do not invent these rules in Phase 2; consume Phase 1.5 semantics.

4B. FIELD-LEVEL SECURITY — BINDING

Phase 2 MUST inspect and consume Phase 1.5 Field-Level Security if it exists.

Mutation side:

A caller being allowed to update an Opportunity does NOT automatically mean the caller may modify every field.

Analyze field-level WRITE authorization for commands such as:

- UpdateOpportunity
- ReassignOpportunity
- ChangePipelineStage
- WinOpportunity
- LoseOpportunity

Examples of potentially protected fields:

- AssignedPrincipal
- EstimatedAmount
- PipelineStage
- lifecycle outcome
- custom fields
- tenant-controlled sensitive fields

Field-level WRITE authorization MUST be evaluated against fields actually being mutated, not merely fields present in the aggregate or request/response type.

Unchanged protected fields MUST NOT cause a write-authorization failure merely because they exist on the aggregate.

The API contract MUST distinguish, where relevant:

- field omitted / unchanged
- field explicitly set to null
- field explicitly cleared/reset
- field set to a new value

Do not allow serialization defaults to silently become field mutations.

Query side:

Record READ permission does NOT automatically imply permission to read every field.

Get/List/detail projections must apply field-level READ authorization where Phase 1.5 requires it.

The frontend MUST NOT receive sensitive fields and merely hide them visually.

Unauthorized fields must be omitted, masked, or projected according to the binding Phase 1.5 policy model.

Do not invent new FLS semantics.

Use exactly what Phase 1.5 established.

4C. POLICY DECISIONS, OBLIGATIONS AND APPROVALS

Do not assume authorization is only:

Allow / Deny.

Inspect Phase 1.5 for richer policy-decision semantics such as:

- Allow
- Deny
- RequireApproval
- obligation
- condition
- limit
- masking requirement
- additional evidence requirement

If Phase 1.5 supports obligations, Phase 2 MUST consume them rather than collapsing them into boolean authorization.

Example:

User may have permission to Win an Opportunity,
but:

Amount > tenant threshold
-> RequireApproval

or:

Changing owner outside own team
-> RequireApproval / Manager scope

or:

Specific field
-> Mask / Deny Write

Do NOT invent approval rules in Phase 2.

The requirement is to preserve and consume the authorization/policy result already defined by Phase 1.5.

If a policy returns RequireApproval, Phase 2 MUST determine the already-established execution contract, for example conceptually:

- command is rejected pending approval,
- approval request is created,
- requested mutation becomes a pending operation,
- execution is suspended until approval,
- another Phase 1.5-defined semantic applies.

Do NOT choose among these options in Phase 2 unless already binding.

If the integration contract between CRM and the Phase 1.5 policy/approval result is unclear:

mark it OPEN DECISION

and stop before implementation.

5. TENANT + SECURITY INVARIANTS

Existing tenant safety must remain intact.

Verify and preserve:

- TenantId on all relevant records
- PostgreSQL RLS
- unprivileged runtime role
- tenant-safe composite foreign keys
- no cross-tenant references
- no cross-module database FK where EntityRef is required
- optimistic concurrency
- row version semantics

The application MUST NOT run with an RLS-bypassing superuser.

If this problem still exists in Host/runtime configuration, include its correction in the Phase 2 prerequisite work.

6. COMMAND EXECUTION STANDARD

For business mutations, preserve the enterprise execution model established earlier.

Conceptually:

Request

v

Authenticated Principal

v

Tenant / Organization Context

v

Coarse Capability / Action Authorization

v

Tenant-safe Aggregate Load, where required

v

Record / Ownership / Team / Territory Policy Evaluation

v

Field-Level WRITE Authorization for the proposed mutation

v

Policy Obligations / Approval Requirements

v

Validate Expected Version according to the established concurrency contract

v

Domain Command

v

Single Transaction

|— Domain State

|— Evidence

|— Outbox

|— Idempotency

v

Commit

v

Response

Do not weaken this model just to create API endpoints quickly.

Section 4A is authoritative for authorization ordering nuances. The sequence above is conceptual, not permission to bypass Phase 1.5 conventions. Create operations and other operations without an existing aggregate may require the established equivalent flow.

7. IDEMPOTENCY

Identify which commands require idempotency.

At minimum, analyze:

- Create
- Win
- Lose
- stage-changing operations
- other retrievable mutations

Use the existing idempotency infrastructure.

Do not create another idempotency table/mechanism.

Same:

Tenant

+ Principal/Caller

+ Operation

+ Idempotency Key

must preserve the semantics already established in the project.

Same key + same request:

-> replay previous result

Same key + different request:

-> reject

7A. IDEMPOTENCY × CONCURRENCY INTERACTION — BINDING

Idempotency and optimistic concurrency MUST be specified together, not independently.

For each idempotent mutation define behavior for:

A) First request:

expectedVersion = 5

```

idempotencyKey = K
-> command executes
-> aggregate becomes version 6
-> response stored

```

B) Exact retry:

```

expectedVersion = 5
idempotencyKey = K

```

same request hash

```

-> return stored result
-> DO NOT run domain command again
-> DO NOT increment version again
-> DO NOT create duplicate Evidence
-> DO NOT create duplicate Outbox event
-> DO NOT publish another logical event

```

C) Same idempotency key, different request:

```

-> reject as idempotency-key reuse conflict

```

D) New idempotency key with stale expectedVersion:

```

-> concurrency conflict
-> do not mutate state
-> do not generate successful-domain evidence/outbox events

```

Explicitly define transaction ordering so that:

```

Domain State
+ Idempotency Record
+ Evidence
+ Outbox

```

remain atomic.

A successful idempotent replay **MUST** be a replay of the original result, not execution of the command against the new aggregate version.

Also define concurrent duplicate behavior when two requests with the same idempotency key arrive while the original request is still in-flight.

Analyze the established infrastructure behavior, for example:

- wait and replay,
- return an in-progress/conflict response,
- database serialization / unique-constraint coordination,
- another already-established mechanism.

Do not invent ad-hoc in-memory locking or a second idempotency mechanism.

8. OPTIMISTIC CONCURRENCY

Opportunity already has or should have a concurrency/version mechanism.

Phase 2 API mutations must expose and enforce it correctly.

Analyze the appropriate API contract.

Example concepts:

```

expectedVersion

```

```

rowVersion

```

```

ETag / If-Match

```

Do not choose one arbitrarily if the repository already established a convention.

The system must prevent:

```

User A loads version 5

```

```

    User B updates -> version 6

```

```

User A writes version 5

```

```

    -> conflict

```

Do NOT silently overwrite.

9. PIPELINE TRANSITION RULES

Pipeline stages are tenant-configurable.

Do NOT implement them as another hard-coded enum.

Phase 2 must determine how ChangePipelineStage validates:

- stage belongs to tenant
- stage belongs to the relevant pipeline
- stage is active
- transition is allowed
- target lifecycle mapping is valid
- pipeline configuration version is respected if versioning exists

Do NOT assume every stage can transition to every other stage.

Use the Phase 1 pipeline model as implemented.

Changing Pipeline Stage MUST NOT implicitly transition Opportunity lifecycle to Won or Lost unless an already-binding Phase 1 decision explicitly establishes that behavior.

WinOpportunity and LoseOpportunity remain explicit canonical lifecycle commands unless a binding decision says otherwise.

A tenant-configured stage name such as "Won", "Lost", "Teklif Verildi", "Closed" or similar MUST NOT by itself become canonical lifecycle semantics.

9A. INITIAL / ENTRY PIPELINE STAGE

Phase 2 MUST explicitly determine how an Opportunity receives its initial Pipeline Stage.

Analyze:

- Does every Pipeline have exactly one entry/default stage?
- Is the entry stage tenant-configurable?
- Is it inherited from a Sector Template?
- Can CreateOpportunity specify another stage?
- Does Draft have a pipeline stage at all?
- Is the entry stage assigned when moving Draft → Open?
- What happens if the configured entry stage is disabled?
- What happens after pipeline configuration/version changes?
- What happens to existing Opportunities that reference a stage later disabled/retired from active configuration?

Historical references MUST NOT be silently destroyed merely because a stage is no longer active for new transitions.

Do NOT choose the first stage by sort order unless this behavior is explicitly binding.

Do NOT hard-code a named stage such as:

"Bekliyor"

"New"

"Qualification"

There must be explicit entry/default-stage semantics if the model requires one.

If Phase 1 did not settle this, mark:

OPEN DECISION - Initial Pipeline Stage Semantics

before implementation.

10. WON / LOST SEMANTICS

Won and Lost are canonical lifecycle outcomes.

They must not merely be arbitrary tenant stage names.

Winning an opportunity should conceptually:

Open

-> Won

Losing:

Open

-> Lost

Determine required information from existing binding decisions.

For Lost, inspect whether we already have:

- structured LostReason
- reason taxonomy
- optional note
- competitor

Do not invent these if not decided.

If the current model only has free-text cancel reason, identify the migration/design consequence.

Do not equate:

Canceled

=

Lost

without reviewing the binding model and existing migration implications.

11. CUSTOMER NEED — DO NOT DESTROY OR IGNORE

CustomerNeed / OpportunityNeed is strategically important.

Existing implementation may currently have it persisted but not behaviorally connected.

Do NOT delete it as dead code.

The target concept is:

Customer Need

-> approximate economic need potential

Example:

Yatak Odası 120,000

Yemek Odası 90,000

Yatak 30,000

Selected needs:

Potential Need Value ≈ 240,000

This is NOT guaranteed revenue.

Keep these concepts semantically distinct:

Need Potential

!=

Opportunity Forecast

!=

Quote Total

!=

Won Revenue

Full AI-driven Customer Need Intelligence may belong to a later phase.

Therefore in Phase 2:

- preserve its model
- do not prematurely implement AI
- do not delete it
- identify any Phase 2 integration point required by Opportunity creation/editing
- report what should remain deferred

Future sources may include:

manual

ai_voice

ai_product_inference

integration

with provenance/confidence/confirmation.

Do not implement future AI fields unless the binding spec or current roadmap requires them now.

11A. UPDATE OPPORTUNITY CAPABILITY

Phase 2 must analyze how editable Opportunity data is modified after creation.

Do NOT assume Create + ChangeStage + Win/Lose is sufficient for a usable CRM.

Determine whether Phase 2 requires an:

UpdateOpportunity

capability or narrower purpose-specific commands.

Inspect fields such as:

- expected / estimated amount
- expected close date
- description
- source
- assigned principal
- customer/party reference where legally mutable
- custom fields

- other CRM-owned mutable attributes

Prefer business-intent commands where a field has meaningful domain semantics.

Do NOT create a generic unrestricted PATCH that bypasses:

- domain invariants
- field-level authorization
- evidence
- concurrency
- policy obligations

Classify every mutable field as:

- editable in Phase 2
- editable through dedicated command
- immutable
- deferred

Also classify domain ownership for each relevant field/reference as applicable:

- CRM-owned
- MasterData / Party-owned or reference-only
- Pipeline-owned
- Security / Authorization-owned
- future Sales-domain-owned
- another existing bounded-context owner

A CRM command MUST NOT mutate data owned by another bounded context merely because the value appears on an Opportunity projection.

12. API DESIGN

After reconciling the domain, propose the HTTP API.

Possible conceptual shape:

POST /opportunities

POST /opportunities/{id}/open

POST /opportunities/{id}/stage

POST /opportunities/{id}/win

POST /opportunities/{id}/lose

But these URLs are NOT binding.

Use the project's established ASP.NET Core API conventions.

For every endpoint specify:

- method
- route
- request contract
- response contract
- authorization capability
- idempotency behavior
- concurrency behavior
- status codes
- validation failures
- domain conflicts
- not-found vs unauthorized behavior

Do not leak tenant data through differences in error responses.

13. QUERY SIDE

Phase 2 is primarily Commands/API, but determine the minimum query capability required to make the flow usable.

Examples may include:

- Get Opportunity
- List Opportunities
- Get available pipeline stages

Do NOT build a reporting platform.

Only include query endpoints necessary for a usable Opportunity workflow.

Clearly separate **Required in Phase 2** vs **Future query/reporting capability**.

13A. AVAILABLE ACTIONS / AVAILABLE TRANSITIONS — BINDING

The frontend **MUST NOT** reproduce or guess backend domain and authorization rules.

Analyze whether Phase 2 requires a query/projection such as conceptually:

AvailableActions

AvailableTransitions

OpportunityCapabilities

Do not assume those exact names.

For an Opportunity, the backend may need to expose the actions currently available to the requesting principal, for example:

- canEdit
- canOpen
- canChangeStage
- allowedTargetStages
- canWin
- canLose
- canReassign

The result **MUST** consider, where applicable:

- lifecycle
- current pipeline stage
- transition rules
- action authorization
- record-level access
- field-level security
- policy obligations
- tenant configuration

The frontend should render actions based on authoritative backend results.

This query is a UX aid and **MUST NOT** replace command-side authorization.

Every command must re-evaluate authorization and invariants at execution time.

Do not collapse richer policy outcomes into misleading boolean UI capabilities.

If an action is available only subject to approval, obligation, masking, limit, or another condition, preserve that distinction according to the existing Phase 1.5 contract.

Determine whether this belongs in Phase 2.

If not, explicitly document how the frontend learns valid actions without duplicating domain logic.

14. EVENTS

Determine the business/integration events required.

Examples conceptually:

OpportunityCreated

OpportunityOpened

OpportunityStageChanged

OpportunityWon

OpportunityLost

OpportunityReassigned

Do not blindly implement all of them.

Determine:

- which are domain events
- which become integration events
- event naming convention
- version
- correlation id
- causation id
- aggregate version
- payload

Use the existing outbox.

No direct message publication inside the business transaction.

15. OUTBOX

Existing current-state analysis found that outbox records were created but no dispatcher existed.

Inspect whether that is still true.

If still missing:

- identify whether dispatcher is a Phase 2 prerequisite
- propose the smallest correct implementation
- preserve at-least-once delivery assumptions
- preserve idempotent consumers
- do not introduce Kafka/MSK/etc just for Phase 2

Use the current infrastructure unless a concrete requirement says otherwise.

16. EVIDENCE / AUDIT

Risk-relevant mutations should preserve business evidence.

Do NOT replace Evidence with ordinary application logs.

For each Phase 2 command determine whether evidence is required.

Evidence should capture sufficient context such as:

- Actor
- Action
- Object
- Tenant
- timestamp
- correlation
- before/after or relevant decision data where required

Use the existing evidence architecture.

17. ERROR MODEL

Define a consistent error taxonomy.

At minimum analyze:

- validation error
- unauthorized
- forbidden
- not found
- concurrency conflict
- illegal lifecycle transition
- invalid pipeline transition
- idempotency conflict
- approval-required / policy-obligation outcome, if represented at API boundary
- field-level write/read restriction behavior
- tenant isolation behavior

Use ProblemDetails or the project's established API error convention.

Do not expose internal implementation details.

18. TEST STRATEGY

Before implementation, define the required tests.

At minimum:

DOMAIN

- valid lifecycle transitions
- illegal transitions
- pipeline transition rules
- initial/entry stage semantics
- inactive/retired stage historical-reference behavior
- stage changes do not implicitly alter canonical lifecycle unless explicitly binding
- Won terminal behavior
- Lost terminal behavior

APPLICATION

- coarse action authorization
- record-level authorization

- field-level WRITE authorization
- policy obligations / approval-required outcomes
- idempotency
- idempotency × concurrency replay behavior
- concurrent duplicate idempotency requests
- evidence
- outbox
- concurrency

INTEGRATION

- PostgreSQL
- RLS with unprivileged runtime role
- tenant isolation
- transaction rollback
- composite tenant-safe relationships

API

- happy paths
- validation errors
- action authorization
- record authorization
- field-level READ projection behavior
- field-level WRITE behavior
- approval/obligation contract where applicable
- authoritative available-actions/transition projection if included
- concurrency conflict
- idempotent retry
- tenant isolation
- status codes

Do not replace real PostgreSQL integration tests with only mocks.

Continue using Testcontainers if that is the established project convention.

19. EXPLICIT NON-GOALS

Unless an existing binding decision says otherwise, do NOT implement in Phase 2:

- Quote
- Quote Version
- Quote public portal
- Quote link tracking
- Sales Order
- Invoice
- Refund
- payment
- full Activity Timeline
- Calls
- Meetings
- Email sync
- Feedback surveys
- KVKK/Consent module
- AI voice processing
- AI need inference
- RAG
- LangGraph
- recommendation agents
- analytics platform
- forecasting engine
- redesigning the Phase 1.5 authorization/policy/approval engine
- inventing new field-security semantics outside Phase 1.5
- rebuilding the pipeline foundation already completed in Phase 1

Do not allow Phase 2 to become "finish the entire CRM".

20. FIRST OUTPUT — NO CODE YET

Before modifying code, create:

docs/architecture-analysis/PHASE_2_OPPORTUNITY_COMMANDS_API_PLAN.md

It must contain:

1. Verified repository state
2. What Phase 0.5 provides to Phase 2
3. What Phase 1 provides to Phase 2
4. What Phase 1.5 provides to Phase 2
5. Current Opportunity aggregate assessment
6. Current vs target lifecycle mapping
7. Pipeline integration
8. Required commands
9. Required queries
10. Proposed API surface
11. Authorization mapping
12. Idempotency strategy
13. Concurrency strategy
14. Evidence requirements
15. Event/outbox requirements
16. Persistence/migration impact
17. Legacy compatibility/migration impact
18. CustomerNeed impact
19. Test plan
20. Exact files expected to change
21. Exact files expected to be added
22. Explicit non-goals
23. Risks / open decisions
24. Recommended implementation sequence
25. Authorization evaluation sequence mapped to actual Phase 1.5 components
26. Field-Level Security READ/WRITE behavior and affected projections/commands
27. Policy obligation / RequireApproval integration contract
28. Initial / entry pipeline stage semantics
29. Mutable-field classification and bounded-context ownership
30. Idempotency × concurrency behavior matrix, including concurrent duplicate requests
31. Available actions / available transitions strategy for frontend consumption
32. Blockers caused by missing binding references, if any

21. OPEN DECISION PROTOCOL

Do NOT silently make product/domain decisions.

If implementation requires a choice that is not already binding, mark it:

OPEN DECISION

For each provide:

- issue
- options
- architectural impact
- migration impact
- your recommendation

But DO NOT implement the recommendation until I approve it.

Examples:

- Draft creation semantics
- whether Create immediately enters Open
- reopening Won/Lost opportunities
- structured LostReason timing
- stage transition matrix behavior
- reassignment rules
- CustomerNeed mutation scope
- API concurrency contract
- Phase 1.5 RequireApproval command integration semantics, if not already binding
- Initial/default pipeline-stage semantics
- behavior for Opportunities referencing retired/inactive stages

- UpdateOpportunity vs purpose-specific update commands
- available-actions/available-transitions API/projection contract

22. STOP CONDITION

After producing the Phase 2 plan:

STOP.

Do not modify production code.

Do not create migrations.

Do not refactor.

Do not implement endpoints.

Do not install packages or developer tooling.

Do not run repository-wide auto-fixes that modify production files.

Do not update lockfiles.

The only intended repository change before approval is the requested Phase 2 planning document, unless an existing repository instruction requires another documentation-only artifact.

Print a concise final summary:

PHASE 2 READINESS

- Existing foundation:
- Required commands:
- Required queries:
- Authorization dependencies:
- Migration impact:
- Critical risks:
- Open decisions:
- Proposed implementation order:
- Ready to implement: YES / NO

Then wait for my approval.